Ryan Joseph Patrick Casalino

Seattle, WA USA | ryan.casalino@gmail.com | 917-282-7506
LinkedIn †linkedin.com/in/ryan-casalino-b2ba55290/ | GitHub | U.S. Person †

Summary

Self-taught Senior Software Engineer with 10 years real world software engineering experience currently specializing in high-availability public infrastructure, containerized microservices and backend systems. Proven track record of managing AWS infrastructure at scale, and expanding and maintaining resilient systems and gRPC APIs. Private-sector experience ranges from startups focused on monetizing Twitch streams to mid-size companies focused on end-to-end payments and ”big data“.

Experience

Ad Hoc, LLC

Senior Software Engineer — Medicare Plan Compare API *Remote* *Feb 2022 – Present*

Backend / Platform (Go, gRPC, AWS, Terraform)

- Shipped **295 merged pull requests** across 4.5 years of sustained delivery, authoring many lines of production **Go** powering CMS's public-facing Medicare.gov Plan Compare experience before wide adoaption of GenAI tools such as Claude Code and Codex. I learned the system by working on it directly and now use these tools to aid my team and I in maintaining the system so as to avoid a great amount of typing.
- Owned end-to-end backend service architecture across **microservices**, building robust **gRPC/protobuf** API layers backed by **PostgreSQL** and orchestrated on **AWS ECS** infrastructure.
- Provisioned and managed cloud infrastructure using **Terraform and Terragrunt**.
- Managed CI/CD jobs using **Groovy and Jenkins**.
- Using **Python and Bash**, streamlined internal maintenance and ETL operations. I oversaw the development to replace a CLI written in Bash to one written in Python with Click (a Python CLI library) thus reducing maintenance burden.
- Day-to-day software tools I used: `docker`, `psql`, `make`.

Sinch *Bothell, WA* *2020 – 2022*

Software Engineer (Ruby, AWS, Automation)

- Engineered an automated, headless video rendering pipeline using Ruby and Bash to dynamically orchestrate AWS EC2 Windows instances executing Adobe After Effects CLI tasks.
- Maintained and refactored complex backend architectures built on Ruby frameworks (including Trailblazer...sadly not Rails...not sure why the Dev who made it picked that...).
- Documented legacy Content Management Systems and championed developer tooling standards (i.e., advocated for `nix`) to streamline onboarding and reduce operational friction (i.e., get the stack up and running quickly).

GawkBox *Seattle, WA* *2017 – 2019*

Software Programmer

- Developed high-performance Go APIs and backend microservices handling CRUD operations and data processing for live-streaming creator monetization platforms.

Liaison Technologies *Alpharetta, GA* *2015 – 2016*

Web Developer

- Engineered responsive user interfaces and client-side application views using AngularJS, SASS, and LESS in close daily collaboration with an international engineering team.
- Built and maintained modular front-end build pipelines using Gulp and Grunt, supporting data-driven CRUD applications backed by NoSQL architectures.

Technical Skills

Languages: Go, Python, Ruby, TypeScript and JavaScript, SQL, Bash / sh, Groovy

Infrastructure & DevOps: AWS (ECS, S3, Aurora), Google Cloud, Terraform, Terragrunt, Docker, Make, Git, Jenkins

Architecture & Services: gRPC, PostgreSQL, REST APIs

Open Source & Personal Projects

Nix/NixOS, C, Perl, et al 2017 - current

- **am_radio**: A command-line internet radio player and discovery tool featuring a vintage tube-radio terminal UI (TUI) mode, written in Perl.

  Interfaces directly with `mpv` as a backend media player, utilizing JSON IPC sockets (`/tmp/am_radio_$$_.sock`) for asynchronous process control, track polling, and status management.

  Integrates with the Radio-Browser.info API via `curl` for global stream searching and leverages `ffprobe` to dump stream codecs and bitrates.

  "Old Time Radio" audio filter injecting a `libavfilter` chain (`highpass=f=300,lowpass=f=4500,acompressor`) to emulate compressed vintage AM sound.

  TUI mode with preset navigation, live tuning, and automated terminal resizing.

- **bss (Boring Static Site Generator)**: A minimalist static site generator written in Perl that pairs Template Toolkit with `rsync` for rapid page generation and asset synchronization.

  An LLM (I cannot remember what model at this point) used a hash map or "pre-scanned template lookup cache" (`%template_map`), optimizing buildtime from $O(n \times m)$ complexity down to $O(n + m)$.

  Features a built-in HTTP server coupled with a detached background file watcher (`setsid()`, `fork()`, and file modification time scanning) that triggers incremental builds upon changes.

  Automatically compiles and writes a JSON metadata file (`__bss_meta.json`) tracking build IDs, timestamps, build duration in milliseconds, page/file counts, and total payload size.

  Driven via a `manifest.ini` configuration file supporting YAML front-matter parsing, Markdown conversion via `Text::Markdown`, content collections, and customizable exclusion rules.

- Configured a Raspberry Pi 4B touchscreen environment running Sway to stream television networks via Chromium kiosk mode and PipeWire audio.
- **Declarative Infrastructure Provisioning:** Authored OpenTofu and Terraform configurations alongside layered Docker images to deploy web applications and ACM SSL certificates across AWS ECS infrastructure.

Education

City University of Seattle *Expected 2027*

Master of Science in Computer Science

School of the Art Institute of Chicago

Bachelor of Fine Arts